

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture - 06

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Solution Without Busy Waiting

Topic

Producer-Consumer Problem

Topic

Reader-Writer Problem

Topics to be Covered



Topic

Reader-Writer Problem

Topic

Dining-Philosopher Problem





Topic : Reader-Writer Problem

Consider a situation where we have a file shared between many people:

- If one of the people tries editing the file, no other person should be reading or writing at the same time, otherwise changes will not be visible to him/her
- However, if some person is reading the file, then others may read it at the same time



Topic : Reader-Writer Problem Solution

- If writer is accessing the file, then all other readers and writers will be blocked
- If any reader is reading, then other readers can read but writer will be blocked

	Reader	writer
Reader	✓	✗
writer	✗	✗



Topic : Reader-Writer Problem Solution

- **Variables:**

- mutex: Binary Semaphore to provide Mutual Exclusion
- wrt: Binary Semaphore to restrict readers and writers if writing is going on
- Readcount: Integer variable, denotes number of active readers

↓
or to block

- **Initialization:**

- mutex: 1
- wrt: 1
- Read count: 0

writers if reading
is going on



Topic : Writer() & Reader() Processes

Writer :-

```
wait(wrt)
// performs writing
signal(wrt)
```

Reader :-

```
wait(mutex)
ReadCount ++ ;
if (ReadCount == 1)
{ wait(wrt); }
signal(mutex)

// performs reading
wait(mutex)
ReadCount -- ;
if (ReadCount == 0)
{ signal(wrt); }
signal(mutex)
```

W1 $\Rightarrow$ writing
R1 $\Rightarrow$ stuck at wait(wrt)
R2
 $\hookrightarrow$ stuck at wait(mutex)



Topic : Dining Philosopher Problem



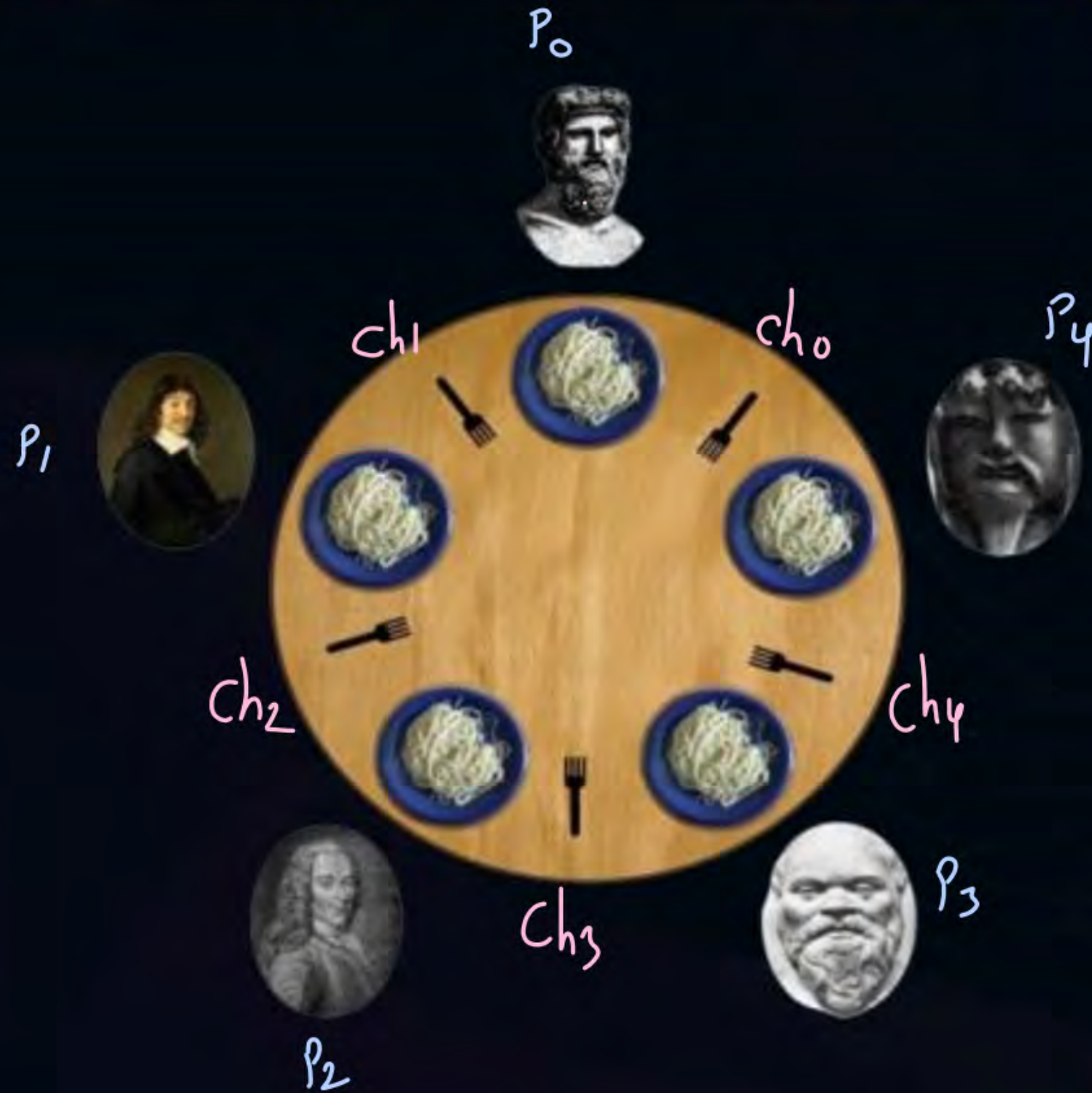


Topic : Dining Philosopher Problem

- K philosophers seated around a circular table
- There is one chopstick between each philosopher
- A philosopher may eat if he can pick up the two chopsticks adjacent to him
- One chopstick may be picked up by any one of its adjacent followers but not both



Topic : Dining Philosopher Problem Solution





Topic : Dining Philosopher Problem Solution

semaphore $ch[k] = \{1, 1, 1, \dots, 1\}$.

Process for philosopher P_i

$wait(ch[i])$

$wait(ch[(i+1) \% k])$

//eating

$signal(ch[i])$

$signal(ch[(i+1) \% k])$

$$\begin{aligned} 1 \% 5 &\Rightarrow 1 \\ 2 \% 5 &\Rightarrow 2 \\ (4+1) \% 5 &\Rightarrow 0 \end{aligned}$$

$i=0$ $ch[0]$
 $ch[1]$

$i=1$ $ch[1]$
 $ch[2]$

$\vdots$

$i=4$ $ch[4]$
 $ch[0]$



Topic : Dining Philosopher Problem Solution



↓
suffers from deadlock



Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table



Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time



Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time
3. One philosopher should pick the left chopstick first and then right chopstick next; while all others will pick the right one first then left one



Happy Learning

THANK - YOU